

Q1. Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

```
mysql> -- BROKEN: comma join with no join condition = Cartesian product
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT E.ENAME, D.DeptName
       -> FROM Employee E, Department D;
```

ENAME	DeptName
Arun	Sales
Arun	Finance
Arun	HR
Arun	IT
Bala	Sales
Bala	Finance
Bala	HR
Bala	IT
Chitra	Sales
Chitra	Finance
Chitra	HR
Chitra	IT
Divya	Sales
Divya	Finance
Divya	HR
Divya	IT
Esakki	Sales
Esakki	Finance
Esakki	HR
Esakki	IT
Farook	Sales
Farook	Finance
Farook	HR
Farook	IT
Gowri	Sales
Gowri	Finance
Gowri	HR
Gowri	IT
Harish	Sales
Harish	Finance
Harish	HR
Harish	IT
Indra	Sales
Indra	Finance
Indra	HR
Indra	IT
Jaya	Sales
Jaya	Finance
Jaya	HR
Jaya	IT

```
40 rows in set (0.000 sec)
```

```
mysql> -- FIXED: proper JOIN with an ON condition  
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT E.ENAME, D.DeptName  
-> FROM Employee E  
-> JOIN Department D ON E.DeptID = D.DeptID;
```

```
+-----+-----+  
| EName | DeptName |  
+-----+-----+  
| Arun  | IT       |  
| Bala  | IT       |  
| Indra  | IT       |  
| Chitra| HR       |  
| Divya | HR       |  
| Jaya  | HR       |  
| Esakki| Finance  |  
| Farook| Finance  |  
| Gowri | Sales    |  
| Harish| Sales    |  
+-----+-----+  
10 rows in set (0.000 sec)
```

Q2. The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

```
mysql> -- BROKEN: uncorrelated - compares every employee to the COMPANY-WIDE
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT EName, Salary, DeptID
-> FROM Employee
-> WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

```
+-----+-----+-----+
| EName | Salary | DeptID |
+-----+-----+-----+
| Arun  | 55000.00 | 1 |
| Chitra | 62000.00 | 2 |
| Esakki | 70000.00 | 3 |
| Indra  | 58000.00 | 1 |
+-----+-----+-----+
4 rows in set (0.009 sec)
```

```
mysql> -- FIXED: correlated sub-query - references the outer row's DeptID
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT E1.EName, E1.Salary, E1.DeptID
-> FROM Employee E1
-> WHERE E1.Salary > (
->     SELECT AVG(E2.Salary)
->     FROM Employee E2
->     WHERE E2.DeptID = E1.DeptID
-> );
```

```
+-----+-----+-----+
| EName | Salary | DeptID |
+-----+-----+-----+
| Arun  | 55000.00 | 1 |
| Chitra | 62000.00 | 2 |
| Esakki | 70000.00 | 3 |
| Gowri | 52000.00 | 4 |
| Indra  | 58000.00 | 1 |
+-----+-----+-----+
5 rows in set (0.000 sec)
```

Q3. A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

```
mysql> -- BROKEN: comma join with no join condition inside a view
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> CREATE OR REPLACE VIEW EmpDeptView_Broken AS
-> SELECT EName, DeptName, Salary
-> FROM Employee, Department;
Query OK, 0 rows affected (0.013 sec)
```

```
mysql> SELECT * FROM EmpDeptView_Broken; -- inflated row count, wrong pairings
```

EName	DeptName	Salary
Arun	Sales	55000.00
Arun	Finance	55000.00
Arun	HR	55000.00
Arun	IT	55000.00
Bala	Sales	48000.00
Bala	Finance	48000.00
Bala	HR	48000.00
Bala	IT	48000.00
Chitra	Sales	62000.00
Chitra	Finance	62000.00
Chitra	HR	62000.00
Chitra	IT	62000.00
Divya	Sales	45000.00
Divya	Finance	45000.00
Divya	HR	45000.00
Divya	IT	45000.00
Esakki	Sales	70000.00
Esakki	Finance	70000.00
Esakki	HR	70000.00
Esakki	IT	70000.00
Farook	Sales	39000.00

```
mysql> -- FIXED: proper join condition inside the view definition
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> CREATE OR REPLACE VIEW EmpDeptView_Fixed AS
-> SELECT E.EName, D.DeptName, E.Salary
-> FROM Employee E
-> JOIN Department D ON E.DeptID = D.DeptID;
Query OK, 0 rows affected (0.008 sec)
```

```
mysql> SELECT * FROM EmpDeptView_Fixed; -- correct row count and pairings
```

EName	DeptName	Salary
Arun	IT	55000.00
Bala	IT	48000.00
Indra	IT	58000.00
Chitra	HR	62000.00
Divya	HR	45000.00
Jaya	HR	NULL
Esakki	Finance	70000.00
Farook	Finance	39000.00
Gowri	Sales	52000.00
Harish	Sales	41000.00

```
10 rows in set (0.001 sec)
```


Q4. Debug the given relational algebra expression that produces incorrect output for a natural join operation.

Broken expression (this is a Cartesian product, not a natural join — it pairs every employee with every department regardless of match):

Employee $\times$ Department

Corrected expression (natural join, matching on the common attribute DeptID):

Employee $\bowtie$ Department

The bug: $\times$ combines every row with every row with no matching condition, producing far more rows than intended and mixing unrelated employee/department pairs. $\bowtie$ only combines rows where the shared attribute (DeptID) is equal.

Q5. The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

```
mysql> -- BROKEN: inserts an employee into a department that does not exist
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO Employee (EmpID, EName, Salary, DeptID)
-> VALUES (111, 'Xavier', 40000, 99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`
(`DeptID`))

mysql> -- FIXED: insert using a DeptID that actually exists
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO Employee (EmpID, EName, Salary, DeptID)
-> VALUES (111, 'Xavier', 40000, 1);
Query OK, 1 row affected (0.009 sec)
```

Q6. Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

```
mysql> -- BROKEN (inefficient): runs a separate subquery for EVERY row of
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.EName,
-> (SELECT D.DeptName FROM Department D WHERE D.DeptID = E.DeptID) AS DeptName
-> FROM Employee E;
+-----+-----+
| EName | DeptName |
+-----+-----+
| Arun  | IT       |
| Bala  | IT       |
| Chitra | HR       |
| Divya | HR       |
| Esakki | Finance  |
| Farook | Finance  |
| Gowri  | Sales    |
| Harish | Sales    |
| Indra  | IT       |
| Jaya  | HR       |
| Xavier | IT       |
| Xavier | IT       |
+-----+-----+
12 rows in set (0.000 sec)
```

```
mysql> -- FIXED (optimized): single JOIN, executed once as a set operation
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.ENAME, D.DeptName
       -> FROM Employee E
       -> JOIN Department D ON E.DeptID = D.DeptID;
+-----+-----+
| EName | DeptName |
+-----+-----+
| Arun   | IT       |
| Bala   | IT       |
| Indra   | IT       |
| Xavier | IT       |
| Xavier | IT       |
| Chitra | HR       |
| Divya  | HR       |
| Jaya   | HR       |
| Esakki | Finance  |
| Farook | Finance  |
| Gowri  | Sales    |
| Harish | Sales    |
+-----+-----+
12 rows in set (0.000 sec)
```

Q7. The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.

```
mysql> -- BROKEN: groups by EName (unique per person) instead of DeptID,
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT DeptID, EName, SUM(Salary) AS TotalSalary
       -> FROM Employee
       -> GROUP BY EName;
ERROR 1055 (42000): Expression #1 of SELECT list is not in GROUP BY clause
in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
mysql> -- FIXED: group by DeptID to get true department-level totals
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT DeptID, SUM(Salary) AS TotalSalary
       -> FROM Employee
       -> GROUP BY DeptID;
+-----+-----+
| DeptID | TotalSalary |
+-----+-----+
| 1      | 241000.00   |
| 2      | 107000.00   |
| 3      | 109000.00   |
| 4      | 93000.00    |
+-----+-----+
4 rows in set (0.013 sec)
```

Q8. Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

```
mysql> -- FIXED: remove the unintended OR condition
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> UPDATE Employee
-> SET Salary = Salary * 1.05
-> WHERE DeptID = 1;
Query OK, 5 rows affected (0.008 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EName  | Salary  | DeptID |
+-----+-----+-----+-----+
| 101   | Arun   | 57750.00 | 1      |
| 102   | Bala   | 50400.00 | 1      |
| 103   | Chitra | 62000.00 | 2      |
| 104   | Divya  | 45000.00 | 2      |
| 105   | Esakki | 70000.00 | 3      |
| 106   | Farook | 39000.00 | 3      |
| 107   | Gowri  | 52000.00 | 4      |
| 108   | Harish | 41000.00 | 4      |
| 109   | Indra   | 60900.00 | 1      |
| 110   | Jaya   | NULL     | 2      |
| 111   | Xavier | 42000.00 | 1      |
| 112   | Xavier | 42000.00 | 1      |
+-----+-----+-----+-----+
12 rows in set (0.000 sec)
```

Q9. Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

```
mysql> -- BROKEN (inefficient): two separate nested sub-queries, each
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT EName
-> FROM Employee
-> WHERE DeptID IN (SELECT DeptID FROM Department WHERE DeptName = 'IT')
-> AND EmpID IN (SELECT EmpID FROM Works_On WHERE ProjID = 1);
+-----+
| EName |
+-----+
| Arun  |
| Bala  |
| Indra  |
+-----+
3 rows in set (0.007 sec)
```



```
mysql> -- FIXED (optimized): single query using JOINS instead of nested
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.ENAME
-> FROM Employee E
-> JOIN Department D ON E.DeptID = D.DeptID
-> JOIN Works_On W ON E.EmpID = W.EmpID
-> WHERE D.DeptName = 'IT' AND W.ProjID = 1;
+-----+
| ENAME |
+-----+
| Arun  |
| Bala  |
| Indra  |
+-----+
3 rows in set (0.001 sec)

mysql> CREATE INDEX idx_employee_deptid ON Employee(DeptID);
Query OK, 0 rows affected (0.022 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_workson_projid ON Works_On(ProjID);
Query OK, 0 rows affected (0.009 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_department_deptname ON Department(DeptName);
Query OK, 0 rows affected (0.011 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Q10. Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

Broken expression: "Find employees who work on at least one project", written incorrectly with a universal quantifier:

$$\{ t \mid t \in \text{Employee} \wedge \forall p \in \text{Project} (\exists w \in \text{Works_On} (w.\text{EmpID} = t.\text{EmpID} \wedge w.\text{ProjID} = p.\text{ProjID})) \}$$

This says *"for every project that exists, the employee must be working on it"* almost no one qualifies, so the result is empty.

Corrected expression (existential quantifier "at least one," not "all"):

$$\{ t \mid t \in \text{Employee} \wedge \exists w \in \text{Works_On} (w.\text{EmpID} = t.\text{EmpID}) \}$$

The bug: using $\forall$ (for all) instead of $\exists$ (there exists) turns "works on some project" into the much stricter "works on every project."